Linked List Templates

Four patterns cover almost every linked list problem.
Consistent naming throughout: `sentinel`, `previous`, `current`, `next`, `slow`, `fast`.

Quick Reference Table

#	Name	Description	Pattern	Sentinel	Key pointers
203	Remove Linked List Elements	Remove all nodes with value == val	Delete/Filter	Yes	`previous`, `current`
83	Remove Duplicates I	Keep one copy of each duplicate value	Delete/Filter	Yes	`previous`, `current`
82	Remove Duplicates II	Remove ALL nodes that have any duplicate	Delete/Filter	Yes	`previous`, `current`
19	Remove Nth From End	Remove nth node from end of list	Delete + fast/slow	Yes	`fast`, `slow`
206	Reverse Linked List	Reverse entire list	Reversal	No	`previous`, `current`
92	Reverse Linked List II	Reverse nodes from position left to right	Reversal (partial)	Yes	`previous`, `current`
25	Reverse Nodes in k-Group	Reverse every k nodes	Reversal (grouped)	Yes	`previous`, `current`
876	Middle of Linked List	Return the middle node	Fast / Slow	No	`slow`, `fast`
141	Linked List Cycle	Detect if cycle exists	Fast / Slow	No	`slow`, `fast`
142	Linked List Cycle II	Return cycle entry node	Fast / Slow	No	`slow`, `fast`
21	Merge Two Sorted Lists	Merge two sorted lists into one	Merge	Yes	`current`
23	Merge K Sorted Lists	Merge k sorted lists using min-heap	Merge + heap	Yes	`current`
143	Reorder List	L0→Ln→L1→Ln-1→... reorder in-place	Find mid + Reverse + Merge	No	`slow`, `fast`, `previous`, `current`
328	Odd Even Linked List	Group odd-index nodes then even-index nodes	Regroup	No	`odd`, `even`
2	Add Two Numbers	Add two numbers represented as reversed linked lists	Sentinel + carry loop; simultaneously advance both lists	**Carry variable**: maintain `carry = sum / 10` across nodes	O(max(m,n))
88	Merge Sorted Array	Merge nums2 into nums1 in-place	Three-pointer from the END; fill backwards to avoid overwriting	**Fill backwards**: start i=m-1, j=n-1, k=m+n-1 to avoid shifting	O(m+n)
160	Intersection of Two Linked Lists	Find the node where two lists intersect	Two pointers: when one reaches end, redirect to other list's head	**Redirect on null**: a = (a==null)? headB:a.next; both travel m+n total	O(m+n)
234	Palindrome Linked List	Check if a linked list is a palindrome	Fast/slow to find middle → reverse second half → compare	**Composite**: fast/slow + reversal + comparison	O(n)

When to Use — Signal → Pattern

Signal in the prompt	Pattern
"remove / delete / skip" nodes by value or duplicate	Delete / Filter (sentinel + `previous`)
"reverse" the list, a sub-range, or in k-groups	Reversal
"middle node", "cycle", "nth node from the end"	Fast / Slow
"merge two sorted lists" (or k lists)	Merge (sentinel + heap for k)
chained steps ("reorder", "palindrome", "sort")	Composite (combine the above)

All Four Templates

```
// 1. DELETE / FILTER  — sentinel guards head removal; previous tracks last kept node
// MENTAL MODEL: previous always points at the last node you decided to keep, so re-wiring it skips anything unwanted.
// WHEN: "remove / delete / skip nodes by value or duplicate"
ListNode sentinel = new ListNode(0);
sentinel.next = head;
ListNode previous = sentinel, current = head;
while (current != null) {
    if (shouldDelete(current)) {
        previous.next = current.next;   // skip node
    } else {
        previous = current;             // keep node, advance previous
    }
    current = current.next;
}
return sentinel.next;

// 2. REVERSAL — re-wire one node at a time
// MENTAL MODEL: flip each arrow to point backward; previous is the growing reversed list trailing behind current.
// WHEN: "reverse the list (whole, partial, or in k-groups)"
ListNode previous = null, current = head;
while (current != null) {
    ListNode next = current.next;   // save before overwrite
    current.next = previous;        // reverse pointer
    previous = current;             // advance
    current = next;
}
return previous;  // new head

// 3. FAST / SLOW — two pointers at different speeds
// MENTAL MODEL: fast covers twice the distance, so when it hits the end slow is exactly halfway; in a loop they must collide.
// WHEN: "middle node", "detect/locate cycle", "nth from end"
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
// slow is at middle  (or cycle detection: slow == fast → cycle found)

// 4. MERGE — sentinel collects nodes from two lists in order
// MENTAL MODEL: repeatedly splice the smaller head onto a growing result tail, like a zipper.
// WHEN: "merge two (or k) sorted lists"
ListNode sentinel = new ListNode(0);
ListNode current = sentinel;
while (a != null && b != null) {
    if (a.val <= b.val) {
        current.next = a;
        a = a.next;
    } else {
        current.next = b;
        b = b.next;
    }
    current = current.next;
}
current.next = (a != null) ? a : b;
return sentinel.next;
```

Pattern 1 — Delete / Filter

`previous` stays on the last **kept** node. When deleting, re-wire `previous.next` to skip `current`. When keeping, advance `previous` to `current`. Always advance `current`.

#203 Remove Linked List Elements

Description: Remove all nodes whose value equals `val`.
Delete condition: `current.val == val`

```
class Solution {
    public ListNode removeElements(ListNode head, int val) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel, current = head;
        while (current != null) {
            if (current.val == val) {
                previous.next = current.next;
            } else {
                previous = current;
            }
            current = current.next;
        }
        return sentinel.next;
    }
}
```

#83 Remove Duplicates from Sorted List

Description: Keep exactly one copy of each value. List is sorted.
Delete condition: `previous != sentinel && previous.val == current.val`

```
class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel, current = head;
        while (current != null) {
            if (previous != sentinel && previous.val == current.val) {
                previous.next = current.next;   // skip duplicate
            } else {
                previous = current;
            }
            current = current.next;
        }
        return sentinel.next;
    }
}
```

#82 Remove Duplicates from Sorted List II

Description: Remove ALL nodes that have duplicate values — only distinct-valued nodes remain.
Key difference from #83: when a duplicate run is detected, skip the entire run (including the first occurrence). Inner `while` advances `current` past the whole run; `previous.next` is rewired to skip all of them.

```
class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel, current = head;
        while (current != null) {
            if (current.next != null && current.val == current.next.val) {
                int dupVal = current.val;
                while (current != null && current.val == dupVal)
                    current = current.next;       // advance past entire run
                previous.next = current;          // skip all duplicates
            } else {
                previous = current;
                current = current.next;
            }
        }
        return sentinel.next;
    }
}
```

#19 Remove Nth Node From End of List

Description: Remove the nth node from the end. Return the head.
Key: `fast` starts n+1 steps ahead of `slow` (both from sentinel). When `fast == null`, `slow` is just before the target node.

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode fast = sentinel, slow = sentinel;
        for (int i = 0; i <= n; i++) fast = fast.next;   // n+1 steps ahead
        while (fast != null) { slow = slow.next; fast = fast.next; }
        slow.next = slow.next.next;                       // delete target
        return sentinel.next;
    }
}
```

Pattern 2 — Reversal

#206 Reverse Linked List

Description: Reverse the entire linked list. Return the new head.
Key: re-wire one node per iteration. `previous` trails behind as the new "next" for each node.

Intuition: flip each node's arrow to point at the node you just came from; `previous` is the reversed list built so far.

```
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode previous = null, current = head;
        while (current != null) {
            ListNode next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        return previous;
    }
}
```

#92 Reverse Linked List II

Description: Reverse only the nodes from position `left` to `right` (1-indexed).
Key: walk `previous` to the node just before `left`. Then insert-at-front `(right - left)` times — each time take `current.next`, unlink it, and attach it right after `previous`.

```
class Solution {
    public ListNode reverseBetween(ListNode head, int left, int right) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel;
        for (int i = 0; i < left - 1; i++) previous = previous.next;
        ListNode current = previous.next;
        for (int i = 0; i < right - left; i++) {
            ListNode next = current.next;
            current.next = next.next;     // unlink next
            next.next = previous.next;    // attach next at front of reversed region
            previous.next = next;
        }
        return sentinel.next;
    }
}
```

#25 Reverse Nodes in k-Group

Description: Reverse every consecutive group of k nodes. Leave remaining nodes (< k) as-is.
Key: `groupPrev` is the tail of the already-processed part. Find the kth node ahead; if it exists, reverse the group using the same insert-at-front technique as #92.

```
class Solution {
    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode groupPrev = sentinel;
        while (true) {
            ListNode kth = getKth(groupPrev, k);
            if (kth == null) break;
            ListNode groupNext = kth.next;
            ListNode previous = groupNext, current = groupPrev.next;
            while (current != groupNext) {
                ListNode next = current.next;
                current.next = previous;
                previous = current;
                current = next;
            }
            ListNode tmp = groupPrev.next;   // will become new tail of this group
            groupPrev.next = kth;            // kth is new head of reversed group
            groupPrev = tmp;
        }
        return sentinel.next;
    }
    private ListNode getKth(ListNode node, int k) {
        while (node != null && k-- > 0) node = node.next;
        return node;
    }
}
```

Pattern 3 — Fast / Slow Pointers

`fast` moves 2× per step. When `fast` reaches the end, `slow` is at the middle.
When `slow == fast` after the start, a cycle is detected.

#876 Middle of Linked List

Description: Return the middle node. For even length, return the second middle.

```
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

#141 Linked List Cycle

Description: Return true if the list contains a cycle.

```
class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}
```

#142 Linked List Cycle II

Description: Return the node where the cycle begins. Return null if no cycle.
Key: after slow meets fast inside the cycle, reset `slow` to `head`. Both then move 1 step at a time — they meet at the cycle entry.

Intuition: the distance from head to the cycle entry equals the distance from the meeting point to the entry, so two 1-step walkers from those spots collide exactly at the entry.

```
class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                slow = head;                          // reset slow to head
                while (slow != fast) {
                    slow = slow.next;
                    fast = fast.next;                 // both move 1 step
                }
                return slow;                          // cycle entry
            }
        }
        return null;
    }
}
```

Pattern 4 — Merge

`sentinel` collects the merged result. `current` is the write pointer.

#21 Merge Two Sorted Lists

Description: Merge two sorted linked lists into one sorted list.

Intuition: like a zipper — always splice on whichever current head is smaller, then advance that list.

```
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        ListNode sentinel = new ListNode(0);
        ListNode current = sentinel;
        while (list1 != null && list2 != null) {
            if (list1.val <= list2.val) {
                current.next = list1;
                list1 = list1.next;
            } else {
                current.next = list2;
                list2 = list2.next;
            }
            current = current.next;
        }
        current.next = (list1 != null) ? list1 : list2;
        return sentinel.next;
    }
}
```

#23 Merge K Sorted Lists

Description: Merge k sorted linked lists into one sorted list.
Key: min-heap of size ≤ k. Always extract the smallest current head; push its next node back.

```
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        ListNode sentinel = new ListNode(0);
        ListNode current = sentinel;
        PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);
        for (ListNode node : lists) if (node != null) pq.offer(node);
        while (!pq.isEmpty()) {
            current.next = pq.poll();
            current = current.next;
            if (current.next != null) pq.offer(current.next);
        }
        return sentinel.next;
    }
}
```

Pattern 5 — Composite (Multi-step)

Problems that chain multiple patterns together.

#143 Reorder List

Description: Reorder list as L0 → Ln → L1 → Ln-1 → L2 → ...
Steps: (1) find middle with slow/fast, (2) reverse second half, (3) merge the two halves.

```
class Solution {
    public void reorderList(ListNode head) {
        // Step 1: find middle
        ListNode slow = head, fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        // Step 2: reverse second half
        ListNode second = slow.next;
        slow.next = null;
        ListNode previous = null, current = second;
        while (current != null) {
            ListNode next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        // Step 3: merge first half and reversed second half
        ListNode first = head;
        second = previous;
        while (second != null) {
            ListNode nextFirst = first.next, nextSecond = second.next;
            first.next = second;
            second.next = nextFirst;
            first = nextFirst;
            second = nextSecond;
        }
    }
}
```

#328 Odd Even Linked List

Description: Group all odd-indexed nodes first, then even-indexed nodes. Preserve relative order within each group. (Indexing is 1-based.)

```
class Solution {
    public ListNode oddEvenList(ListNode head) {
        if (head == null) return null;
        ListNode odd = head, even = head.next, evenHead = even;
        while (even != null && even.next != null) {
            odd.next = even.next;   odd = odd.next;
            even.next = odd.next;   even = even.next;
        }
        odd.next = evenHead;
        return head;
    }
}
```

Pattern Summary

Pattern	Sentinel	Naming	When to use
Delete / Filter	Yes	`previous`, `current`	Remove or skip nodes by condition
Reversal	No (Yes for partial)	`previous`, `current`, `next`	Re-wire pointer direction
Fast / Slow	No	`slow`, `fast`	Middle, cycle, nth from end
Merge	Yes	`current`	Combine two or more lists
Composite	Both	Mix of above	Reorder, sort, rearrange

Sentinel Cheat Sheet

SENTINEL: dummy node before head so head removal needs no special-casing; use it whenever the head itself might be removed/replaced.

```
// Always use sentinel when the head itself might be removed or replaced:
ListNode sentinel = new ListNode(0);
sentinel.next = head;
// ... operations ...
return sentinel.next;   // head may have changed; sentinel.next is always correct
```

82 vs 83 — Side by Side

```
                      #83 (keep one)              #82 (remove all)
Delete condition:     previous.val == current.val  current.val == current.next.val
Action on match:      skip current only            inner while to skip entire run
previous advances?:   only when keeping            only when not a dup
```

#2 Add Two Numbers

Description: Two non-empty linked lists represent non-negative integers stored in reverse order. Add and return the sum as a linked list in reverse order.

Algorithm: Use sentinel + carry variable. Traverse both lists simultaneously; at each step compute `sum = l1.val + l2.val + carry`. Create a new node with `sum % 10`, carry `sum / 10` forward. Continue while either list has nodes or carry is non-zero.

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode sentinel = new ListNode(0);
        ListNode current = sentinel;
        int carry = 0;
        while (l1 != null || l2 != null || carry != 0) {   // ← VARIATION: loop until carry clears
            int sum = carry;
            if (l1 != null) { sum += l1.val; l1 = l1.next; }
            if (l2 != null) { sum += l2.val; l2 = l2.next; }
            carry = sum / 10;                               // ← VARIATION: propagate carry
            current.next = new ListNode(sum % 10);
            current = current.next;
        }
        return sentinel.next;
    }
}
```

Time O(max(m, n)) | **Space** O(max(m, n))

#88 Merge Sorted Array

Description: Merge `nums2` into `nums1` in-place. `nums1` has enough space at the end. Both arrays are sorted.

Algorithm: Fill from the END backwards using three pointers `i` (end of nums1 values), `j` (end of nums2), `k` (end of merged array). This avoids overwriting unprocessed elements in nums1.

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1, j = n - 1, k = m + n - 1;          // ← VARIATION: three pointers from END
        while (i >= 0 && j >= 0)
            nums1[k--] = (nums1[i] >= nums2[j]) ? nums1[i--] : nums2[j--];
        while (j >= 0) nums1[k--] = nums2[j--];            // ← copy remaining nums2
    }
}
```

Time O(m + n) | **Space** O(1)

#234 Palindrome Linked List

Description: Check if a singly linked list is a palindrome in O(n) time and O(1) space.

Algorithm: Composite — (1) fast/slow to find middle, (2) reverse the second half in-place, (3) compare both halves. Variation: combines three separate patterns.

```
class Solution {
    public boolean isPalindrome(ListNode head) {
        // Step 1: find middle using fast/slow
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        // Step 2: reverse second half                    // ← VARIATION: in-place reverse
        ListNode previous = null, current = slow;
        while (current != null) {
            ListNode next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        // Step 3: compare halves
        ListNode i = head, j = previous;
        while (j != null) {
            if (i.val != j.val) return false;
            i = i.next;
            j = j.next;
        }
        return true;
    }
}
```

Time O(n) | **Space** O(1)

#160 Intersection of Two Linked Lists

Description: Find the node at which two singly linked lists intersect. Return null if they do not intersect.

Algorithm: Two pointers `a` and `b` start at `headA` and `headB`. When either reaches the end, redirect it to the other list's head. After at most `m + n` steps they meet at the intersection (or both become null for no intersection). This works because both pointers travel the same total distance.

```
class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        ListNode a = headA, b = headB;
        while (a != b) {
            a = (a == null) ? headB : a.next;   // ← VARIATION: redirect to other list on null
            b = (b == null) ? headA : b.next;
        }
        return a;   // either the intersection node, or null
    }
}
```

Time O(m + n) | **Space** O(1)